# PRINT3D MANAGER DESENVOLVIMENTO DE UM ERP PARA EMPRESAS DE IMPRESSÃO 3D COM APOIO DE AGENTES DE INTELIGÊNCIA ARTIFICIAL

João Antônio Oliveira Barroso[1]

Felipe Dallilo Diniz[2]

**Resumo:** Este trabalho apresenta o desenvolvimento do Print3D Manager ERP e analisa o uso de agentes de inteligência artificial no processo de construção do sistema. A pesquisa é aplicada e descritiva, baseada em um estudo de caso documental do próprio projeto. Foram examinados o código-fonte, o arquivo de contexto, as configurações de doze subagentes do Claude Code e os relatórios de auditoria preservados no repositório. A aplicação reúne cadastros, orçamentos, pedidos, produção, estoque e financeiro para empresas de impressão 3D. Os agentes receberam instruções sobre especialidade, ferramentas permitidas, arquivos de referência e verificações necessárias. O estudo descreve essa configuração e apresenta trechos dos prompts persistentes, além de dois episódios registrados na auditoria: uma condição de corrida no faturamento e uma divergência na avaliação do limite de requisições. Os registros indicam que a divisão de responsabilidades apoiou a revisão do sistema, mas também mostram que testes aprovados e concordância entre agentes não bastam para assegurar a correção do produto. As evidências permitem discutir o processo adotado, sem demonstrar ganho de produtividade ou superioridade em relação a outras formas de desenvolvimento.

**Palavras-chave:** Inteligência artificial. Agentes de software. ERP. Impressão 3D.

[1] Graduando do Curso de Sistemas de Informação da Universidade de Araraquara - UNIARA. Araraquara-SP. E-mail: jaobarroso1@uniara.edu.br.

[2] Orientador. Docente do Curso de Sistemas de Informação da Universidade de Araraquara - UNIARA. Araraquara-SP. E-mail: fddallilo@uniara.edu.br.

# PRINT3D MANAGER DEVELOPMENT OF AN ERP FOR 3D PRINTING COMPANIES WITH SUPPORT FROM ARTIFICIAL INTELLIGENCE AGENTS

**Abstract:** This paper presents the development of Print3D Manager ERP and examines the use of artificial intelligence agents during its construction. The research is applied and descriptive, based on a documentary case study of the project. The evidence includes source code, a project context file, the configuration of twelve Claude Code subagents, and audit reports preserved in the repository. The application integrates customer and resource records, quotes, orders, production, inventory, and finances for 3D printing companies. Agent instructions specify responsibilities, available tools, reference files, and required checks. The study describes these configurations and presents excerpts from persistent prompts, together with two documented audit episodes: a race condition in billing and a disagreement over rate limiting. The records indicate that specialized responsibilities supported software review, while also showing that passing tests and agreement among agents do not establish product correctness. The findings describe the process adopted in this project; they do not demonstrate productivity gains or superiority over other development approaches.

**Keywords:** Artificial intelligence. Software agents. ERP. 3D printing.

# 1 INTRODUÇÃO

Ferramentas de inteligência artificial passaram a integrar atividades de programação, revisão e teste de software. Algumas permitem consultar arquivos, editar o repositório e executar comandos durante uma mesma tarefa. Nesses ambientes, a qualidade da resposta depende tanto da capacidade do modelo quanto das instruções recebidas e das verificações realizadas sobre o código produzido.

O presente trabalho investiga esse uso no desenvolvimento do Print3D Manager ERP, uma aplicação web para empresas de impressão 3D. O sistema foi concebido para reunir informações que participam de uma mesma operação: o cliente solicita uma peça, o orçamento considera materiais e custos, o pedido acompanha a produção e os registros de estoque e financeiro refletem essa execução. A relação entre esses módulos oferece situações em que uma mudança local pode afetar outras partes da aplicação.

Para apoiar o desenvolvimento, foram definidos doze subagentes no Claude Code. Cada um recebeu um papel, um conjunto de ferramentas e instruções persistentes. A configuração foi registrada em arquivos Markdown no diretório .claude/agents. O arquivo PROJECT_CONTEXT.md reuniu decisões técnicas, regras de negócio e orientações de continuidade. Os relatórios em .claude/audits registraram análises especializadas e sua consolidação.

A questão que orienta o estudo é: de que forma agentes especializados podem apoiar o desenvolvimento de um ERP para impressão 3D, considerando a manutenção de contexto, a implementação de regras e a identificação de falhas? O objetivo geral é desenvolver o sistema e analisar o papel dos agentes nesse processo. Os objetivos específicos são apresentar os conceitos necessários ao estudo; descrever a configuração e as instruções dos agentes; documentar o fluxo de trabalho; e discutir suas contribuições e limitações com base nos registros do projeto.

O interesse do estudo está no processo de desenvolvimento assistido por IA. O ERP constitui o artefato em que esse processo foi aplicado. Não se pretende atribuir aos agentes a responsabilidade pelas decisões do projeto nem afirmar que sua utilização tornou o desenvolvimento mais rápido. Essas conclusões exigiriam medidas e comparações que não foram realizadas neste estudo.

# 2 REVISÃO BIBLIOGRÁFICA

## 2.1 Análise de requisitos

A análise de requisitos busca compreender os serviços que o sistema deve oferecer e as condições em que precisa operar. Sommerville (2011) distingue requisitos funcionais, relacionados ao comportamento esperado, de requisitos não funcionais, que expressam restrições e atributos de qualidade. Essa distinção ajuda a separar uma necessidade de negócio, como calcular um orçamento, de exigências como restringir o acesso a informações financeiras.

Pressman (2011) relaciona a compreensão do problema à construção e à avaliação do software. Requisitos não se limitam a uma lista inicial: precisam ser examinados quando surgem dúvidas sobre dados, estados ou exceções de um fluxo. Essa necessidade permanece no desenvolvimento assistido por IA, pois a instrução enviada ao agente também depende de uma interpretação do problema.

## 2.2 Arquitetura de software

A arquitetura organiza os elementos do sistema e suas relações. Bass, Clements e Kazman (2021) discutem como essas decisões afetam atributos de qualidade e impõem escolhas ao desenvolvimento. A definição de limites entre módulos, por exemplo, influencia a localização das regras e o alcance de uma alteração.

Fowler (2002) descreve a camada de serviços como um limite para as operações da aplicação. Gamma et al. (2000) apresentam padrões que permitem tratar problemas recorrentes de projeto. Esses conceitos oferecem critérios para analisar o código gerado por um agente: uma funcionalidade pode funcionar isoladamente e, ainda assim, distribuir responsabilidades de forma incompatível com a organização adotada pelo sistema.

## 2.3 Persistência de dados

O modelo relacional oferece mecanismos para representar dados e preservar relações entre eles. Date (2004) e Elmasri e Navathe (2016) abordam chaves, restrições e transações como elementos da consistência do banco. Em aplicações de gestão, a correção depende também de como operações simultâneas modificam esses dados.

Uma consulta seguida de uma inserção, por exemplo, não garante por si só que outra execução não realize a mesma operação no intervalo. Por isso, a análise da persistência precisa considerar as garantias do banco e das transações, além do resultado observado em uma execução sequencial.

## 2.4 Segurança de software

Stallings (2017) apresenta confidencialidade, integridade e disponibilidade como objetivos de segurança. Na aplicação web, autenticação e autorização participam dessa proteção, juntamente com o tratamento de entradas, o armazenamento de segredos e a configuração dos componentes que recebem as requisições.

A análise de segurança depende do caminho real de acesso à aplicação. Um mecanismo de controle pode parecer adequado quando examinado no código e apresentar outro comportamento diante da configuração de rede. Esse aspecto justifica observar conjuntamente a aplicação e sua infraestrutura, em vez de limitar a revisão a uma classe ou arquivo.

## 2.5 Testes de software

Testes permitem confrontar o comportamento implementado com o comportamento esperado. Sommerville (2011) e Pressman (2011) discutem verificações em diferentes níveis, desde unidades de código até a integração entre componentes. A escolha dos cenários define quais comportamentos serão efetivamente observados.

Uma suíte aprovada informa que os casos executados atenderam às asserções definidas. Ela não permite concluir que todos os estados possíveis foram examinados. Cenários de concorrência, autorização e falha de integração exigem preparação específica e podem permanecer ausentes mesmo quando há testes para o fluxo principal.

## 2.6 Modelos de linguagem

Modelos de linguagem produzem respostas a partir das instruções e do contexto disponibilizados. Em ferramentas de programação, esse contexto pode incluir o pedido do desenvolvedor, arquivos do projeto e resultados de comandos. O ambiente de execução amplia as ações possíveis, mas não transforma uma resposta plausível em evidência de correção.

Peng et al. (2023) estudaram uma tarefa delimitada de implementação de um servidor HTTP em JavaScript com apoio do GitHub Copilot. O grupo com acesso à ferramenta concluiu a tarefa 55,8% mais rapidamente. O resultado se refere àquelas condições experimentais e não deve ser transferido diretamente para o desenvolvimento de um ERP completo.

### 2.7 Engenharia de prompt

Neste estudo, prompt designa a instrução que orienta a atuação do modelo. A engenharia de prompt compreende a definição do objetivo, dos dados de entrada, dos limites de atuação e do resultado esperado. Em um ambiente com acesso ao repositório, isso inclui indicar quais fontes consultar e quais verificações executar.

Há uma diferença entre a solicitação de uma tarefa e o prompt persistente de um agente. A primeira descreve uma necessidade do momento; o segundo estabelece orientações reutilizadas em várias tarefas. A documentação de subagentes do Claude Code descreve arquivos que combinam metadados de configuração e instruções para o papel especializado (ANTHROPIC, 2026).

### 2.8 Sistemas multiagente

Sistemas multiagente distribuem atividades entre agentes que participam de um processo comum. No desenvolvimento de software, a especialização pode acompanhar papéis como arquitetura, implementação e testes. Hong et al. (2024) apresentam o MetaGPT, que associa papéis a procedimentos de trabalho para organizar a colaboração.

Qian et al. (2024) apresentam o ChatDev, no qual agentes participam de etapas de projeto, codificação e teste por meio de comunicação estruturada. Esses trabalhos ajudam a compreender a divisão de responsabilidades, mas não constituem a implementação utilizada neste TCC. O projeto empregou subagentes configurados no Claude Code, com encaminhamento de tarefas e supervisão do desenvolvedor.

Os estudos citados mostram formas de organizar a colaboração. Para avaliar o caso do Print3D Manager, é necessário examinar como essa organização foi concretizada: quais instruções foram registradas, quais ferramentas cada papel podia utilizar e quais evidências foram exigidas antes de aceitar uma entrega.

## 3 DESENVOLVIMENTO

### 3.1 Procedimento do estudo

A pesquisa é aplicada, de caráter descritivo, e utiliza um estudo de caso do desenvolvimento do Print3D Manager ERP. A análise documental considera quatro grupos de evidências: código-fonte e configuração da aplicação; arquivo PROJECT_CONTEXT.md; definições dos agentes em .claude/agents; e relatórios em .claude/audits. As telas apresentadas em Resultados ilustram funcionalidades do produto e não constituem uma avaliação com usuários.

O procedimento consistiu em identificar as responsabilidades previstas para cada agente, examinar suas instruções persistentes e relacioná-las aos registros de auditoria. Foram selecionados dois episódios que permitem acompanhar problemas concretos: o risco de duplicação de receita e a reavaliação de uma falha no limite de requisições. A análise distingue o que foi prescrito nos prompts daquilo que aparece documentado nos relatórios.

O repositório não reúne, em um único registro completo, todas as mensagens enviadas em todas as sessões, versões do modelo, parâmetros de inferência e custos. Assim, os exemplos de prompts apresentados adiante são trechos dos arquivos persistentes efetivamente disponíveis. Não são uma reconstrução de conversas históricas. Da mesma forma, os resultados da auditoria são tratados como registros daquela rodada, e não como uma nova certificação do estado atual do sistema.

### 3.2 Análise da solução

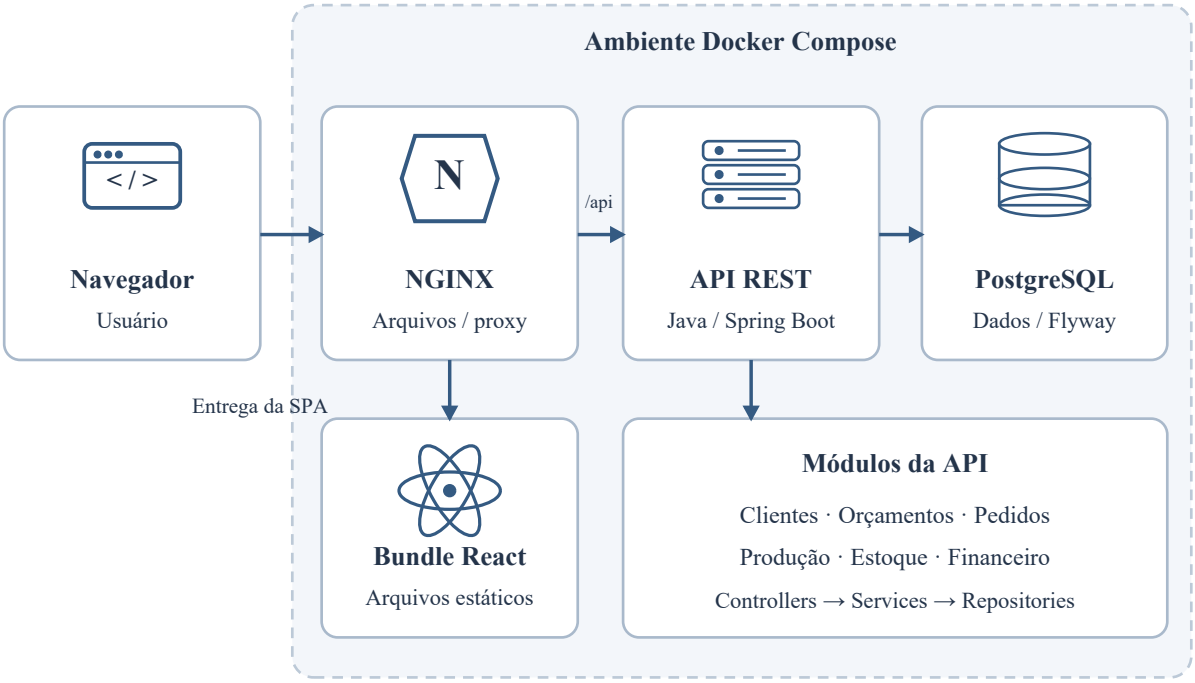
O sistema foi organizado a partir do fluxo de uma empresa de impressão 3D. Os cadastros de clientes, impressoras e filamentos fornecem dados para o orçamento e para a produção. O pedido reúne os itens solicitados e seu acompanhamento; o histórico de impressões registra a execução; e o financeiro acompanha receitas e despesas.

A precificação considera filamento, energia, hora de máquina, desgaste e margem. Essa regra exige coerência entre unidades, quantidades e parâmetros da impressora. Também existem regras que atravessam módulos, como a conversão de orçamento em pedido e o vínculo de uma receita ao faturamento. Essas dependências orientaram a divisão das tarefas entre agentes técnicos e agentes de domínio.

### 3.3 Arquitetura da aplicação

O Print3D Manager utiliza React com TypeScript na interface e Java com Spring Boot na API. O backend foi organizado como monólito modular, com pacotes por funcionalidade. Dentro de cada domínio, controllers recebem requisições, services concentram operações da aplicação e repositories realizam o acesso aos dados. DTOs delimitam os contratos de entrada e saída.

Figura 1 - Arquitetura do Print3D Manager ERP




Fonte: elaboração própria com base no código e na configuração do projeto, 2026.

Na Figura 1, o navegador carrega a aplicação React a partir do NGINX. As chamadas ao caminho /api são encaminhadas ao Spring Boot, que acessa o PostgreSQL. O Flyway aplica migrações ao banco. Docker Compose organiza os serviços de NGINX/frontend, backend e banco; ele não participa como uma etapa sequencial do processamento de cada requisição. Os módulos de negócio pertencem à API, e não ao trajeto entre o banco e o navegador.

Tabela 1 - Tecnologias utilizadas

| **Camada** | **Tecnologias utilizadas** |
|---|---|
| Backend | Java 21, Spring Boot 3.5, Spring Security, JWT, Spring Data JPA, Hibernate, Flyway e MapStruct. |
| Banco de dados | PostgreSQL 16. |
| Frontend | React 19, TypeScript, Vite, Material UI, TanStack Query, React Hook Form, Zod e Recharts. |
| Infraestrutura | Docker, Docker Compose e NGINX. |
| Testes | JUnit 5, Mockito, Testcontainers e MockMvc. |

Fonte: código e configurações do projeto, 2026.

O uso de Flyway mantém as alterações do esquema em arquivos versionados. A configuração por variáveis de ambiente separa parâmetros de execução do código. A autenticação utiliza JWT, e a autorização restringe operações por perfil. Esses elementos também aparecem nos prompts como condições que os agentes precisam respeitar ao propor mudanças.

### 3.4 Configuração dos agentes

Os doze agentes foram definidos em arquivos Markdown no diretório .claude/agents. Cada arquivo começa com um cabeçalho YAML. O campo name identifica o agente; description descreve as situações em que deve ser utilizado; e tools lista as ferramentas disponíveis. Após o cabeçalho, o texto define missão, referências, escopo, limites e procedimento de trabalho.

**Quadro 1 - Campos selecionados do cabeçalho de tech-lead.md**

```
---
name: tech-lead
tools: Read, Grep, Glob, Bash, PowerShell
---
```

Fonte: .claude/agents/tech-lead.md. Campo description omitido neste recorte.

O Quadro 1 reproduz o cabeçalho de tech-lead.md. Sua lista não inclui Edit ou Write. Essa diferença também aparece nas definições de security, performance e auditor-final, que concentram a análise e o relato. Agentes de implementação, como backend e frontend, incluem ferramentas de edição. A disponibilidade de Bash e PowerShell, contudo, também permite executar comandos; portanto, a ausência de Edit e Write não deve ser interpretada como isolamento de segurança suficiente para impedir toda escrita. Os limites dependem também das permissões e da supervisão do ambiente.

Tabela 2 - Agentes configurados no projeto

| Agente | Responsabilidade definida | Atuação |
|---|---|---|
| tech-lead | Arquitetura, impacto e encaminhamento das tarefas. | Análise |
| backend | API, serviços, DTOs e padrões dos módulos. | Implementação |
| frontend | Páginas, formulários e integração com a API. | Implementação |
| database | Modelo de dados, JPA, SQL e migrações Flyway. | Implementação |
| erp | Orçamentos, pedidos, estados e faturamento. | Implementação |
| printing3d | Material, impressoras e custos de impressão. | Implementação |
| security | Autenticação, autorização e vulnerabilidades. | Auditoria |
| performance | Consultas, índices e gargalos de interface. | Análise |
| qa | Testes e identificação de lacunas de cobertura. | Testes |
| devops | Docker, NGINX e configuração dos ambientes. | Infraestrutura |
| tcc | Coerência da documentação com o repositório. | Documentação |
| auditor-final | Consolidação de evidências e veredito. | Auditoria |

Fonte: definições em .claude/agents, 2026.

A Tabela 2 apresenta responsabilidades definidas nos arquivos. A especialização é estabelecida por instruções e ferramentas; não corresponde ao treinamento de doze modelos próprios. Os cabeçalhos examinados não fixam um campo model. Por isso, os arquivos não bastam para identificar qual versão de modelo foi utilizada em cada execução ou se todos os papéis utilizaram a mesma versão.

### 3.5 Contexto do projeto

O arquivo PROJECT_CONTEXT.md foi utilizado como referência para retomar o trabalho entre sessões. Ele reúne a stack, decisões de arquitetura, padrões de código, etapas de implementação e regras de negócio. Os prompts determinam sua leitura antes da atuação e pedem que pontos decisivos sejam confirmados no código.

Esse mecanismo não equivale a uma memória automática de todas as conversas. Trata-se de um documento que precisa ser atualizado e consultado. Se o texto estiver desatualizado, pode orientar uma resposta incorreta. A própria configuração de tech-lead alerta para a diferença entre um pacote citado na documentação e sua existência no repositório, exigindo confirmação antes de assumir que uma funcionalidade foi implementada.

O contexto comum oferece uma referência para decisões que atravessam especialidades. Para uma mudança em orçamento, por exemplo, backend precisa conhecer o contrato da API, database precisa verificar a persistência e erp e printing3d precisam analisar as regras. O arquivo reduz a dependência de informações espalhadas em conversas, mas não elimina a necessidade de ler os módulos envolvidos.

### 3.6 Instruções persistentes

Os prompts foram organizados para definir uma atuação verificável. Em backend.md, a orientação combina um padrão de implementação com comandos de compilação e teste. Em qa.md, a instrução proíbe alterar o código de produção apenas para fazer um teste passar. Em auditor-final.md, a revisão exige evidências para sustentar o veredito. O Quadro 2 reúne trechos literais desses arquivos, preservando a natureza de instrução.

**Quadro 2 - Trechos dos prompts persistentes**

| |
|---|
| **backend.md - contrato de implementação** |
| DTOs sempre como **Java Records**; controllers nunca recebem/retornam entidades JPA. |
| **backend.md - preservação do código** |
| Faça alterações mínimas e focadas no problema solicitado. |
| **qa.md - execução e evidência** |
| Rode cd backend && .\mvnw.cmd test (suíte completa exige Docker) e reporte o resultado real, nunca assuma que passou sem rodar. |
| **auditor-final.md - revisão independente** |
| Não faça auditoria superficial "de carimbo" — cada item do checklist precisa de evidência (comando rodado, arquivo lido), não suposição. |

Fonte: arquivos backend.md, qa.md e auditor-final.md em .claude/agents.

Essas instruções tornam explícito o que deve acompanhar uma entrega. O relatório de backend pede arquivos alterados, padrão de referência e resultado de build e testes. O de segurança solicita localização, cenário de exploração, recomendação e agente responsável. O de QA pede testes executados, lacunas e bugs encontrados. O formato comum de severidade facilita a consolidação, embora a classificação ainda dependa da análise do cenário.

Também existem encaminhamentos entre especialidades. O agente backend deve alinhar mudanças de schema com database e regras de negócio com erp ou printing3d. Essas orientações estabelecem o processo pretendido; a presença delas no arquivo, isoladamente, não prova que todos os encaminhamentos ocorreram em todas as tarefas. Os

relatórios preservados permitem examinar partes desse processo, sobretudo a rodada de auditoria.

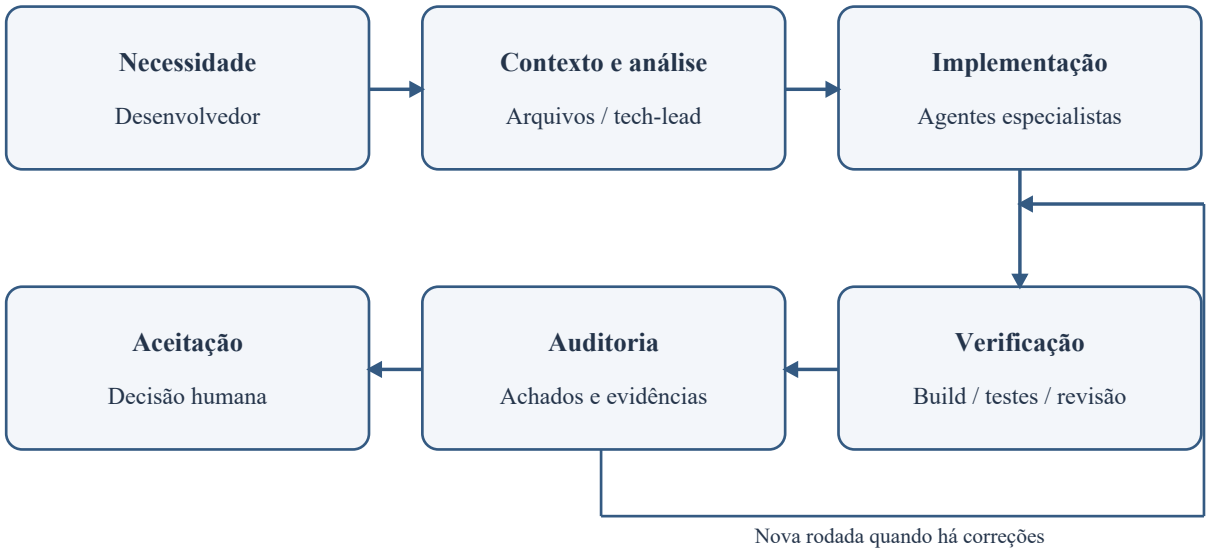
O autor também utilizou IA como apoio à revisão acadêmica. No prompt de revisão fornecido para este estudo, solicitou análise de ortografia, escrita acadêmica, clareza, coesão, conteúdo técnico e normas da ABNT, seguida de uma versão revisada do trecho. A instrução incluía preservar o significado e não inventar informações. Esse uso editorial é distinto da atuação dos agentes de desenvolvimento e não constitui evidência das solicitações utilizadas para construir o ERP.

Como as mensagens originais de implementação não foram preservadas pelo autor, esta descrição utiliza os prompts persistentes encontrados no projeto. A configuração permite examinar orientações reutilizáveis, mas não recuperar integralmente as tarefas enviadas em cada sessão. A revisão acadêmica assistida também exige conferência das fontes, pois uma sugestão de correção bibliográfica não substitui a identificação da edição efetivamente consultada.

### 3.7 Fluxo de trabalho

O fluxo adotado começa com uma necessidade definida pelo desenvolvedor. O contexto e os arquivos relevantes são consultados, a tarefa é atribuída ao papel adequado e a proposta é examinada por meio de código, comandos e revisão. Quando a alteração afeta mais de uma área, o plano de tech-lead indica os especialistas e a ordem de atuação. As decisões de aceitação continuam sob responsabilidade do desenvolvedor.

Figura 2 - Fluxo de desenvolvimento assistido por agentes




Fonte: elaboração própria com base nas instruções persistentes do projeto, 2026.

A Figura 2 representa o fluxo prescrito nos arquivos, sem afirmar que houve execução simultânea dos doze agentes. O compartilhamento de contexto ocorre pelo repositório e pelos relatórios. Os agentes de análise devolvem recomendações; os responsáveis pela implementação podem aplicar alterações em uma rodada seguinte. Depois, os cenários relevantes precisam ser verificados novamente.

Há particularidades na persistência das saídas. security e auditor-final devolvem seus relatórios para que o responsável pela invocação os salve. Outros agentes são orientados a gravar o relatório em .claude/audits. Esses arquivos representam a última execução registrada de cada papel; o histórico anterior depende do versionamento. Essa característica limita a reconstrução cronológica quando não há consulta aos commits correspondentes.

### 3.8 Critérios de verificação

As instruções exigem compilação do backend com Maven Wrapper, testes de comportamento e compilação do frontend. O auditor final deve verificar resultados reais, consultar relatórios especializados e distinguir itens verificados, não verificados e não aplicáveis. A qualidade da evidência importa mais do que a frase de conclusão produzida pelo agente.

No domínio financeiro, verificar um pedido em execução sequencial não substitui um teste com duas requisições simultâneas. Na segurança, observar a autorização de uma rota não substitui examinar o caminho de rede usado para acessá-la. Esses exemplos orientam a interpretação dos resultados: a revisão especializada acrescenta perguntas ao processo, mas suas respostas precisam ser confrontadas com o ambiente e com os cenários executados.

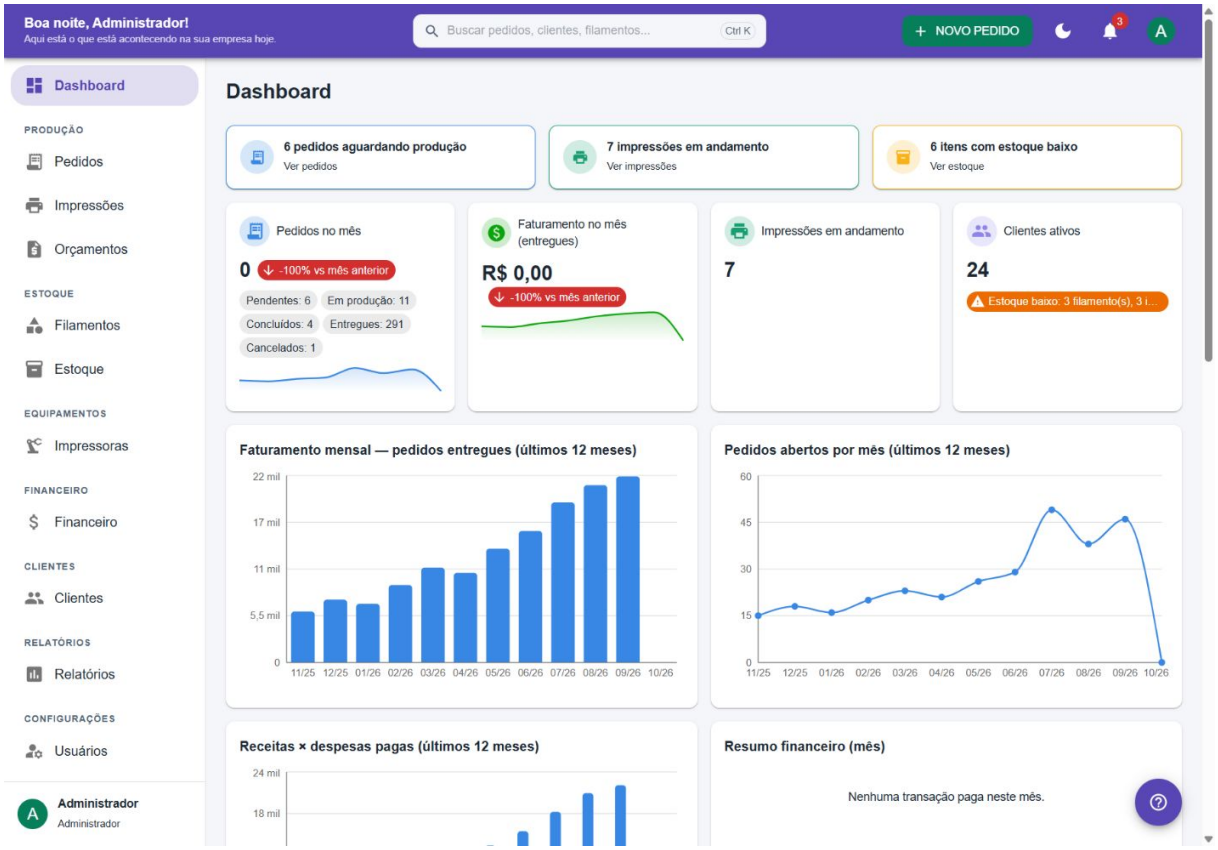
## 4 RESULTADOS

### 4.1 Produto desenvolvido

O produto resultante é uma aplicação web que integra cadastros, orçamentos, pedidos, histórico de impressões, estoque, dashboard, financeiro e relatórios. A interface permite acompanhar registros da operação, enquanto a API concentra regras e persistência. As configurações de custo das impressoras fornecem parâmetros para a precificação.

As Figuras 3 a 5 apresentam telas do produto em funcionamento, capturadas em modo claro em 14 de outubro de 2026. Os dados exibidos pertencem ao ambiente de demonstração e não representam desempenho financeiro ou operacional de uma empresa real. O roteiro de carga disponível em scripts/demo identifica os registros como fictícios e preparados para apresentação do sistema. Como a carga contém registros de meses anteriores, indicadores referentes ao mês da captura podem apresentar valores zerados.

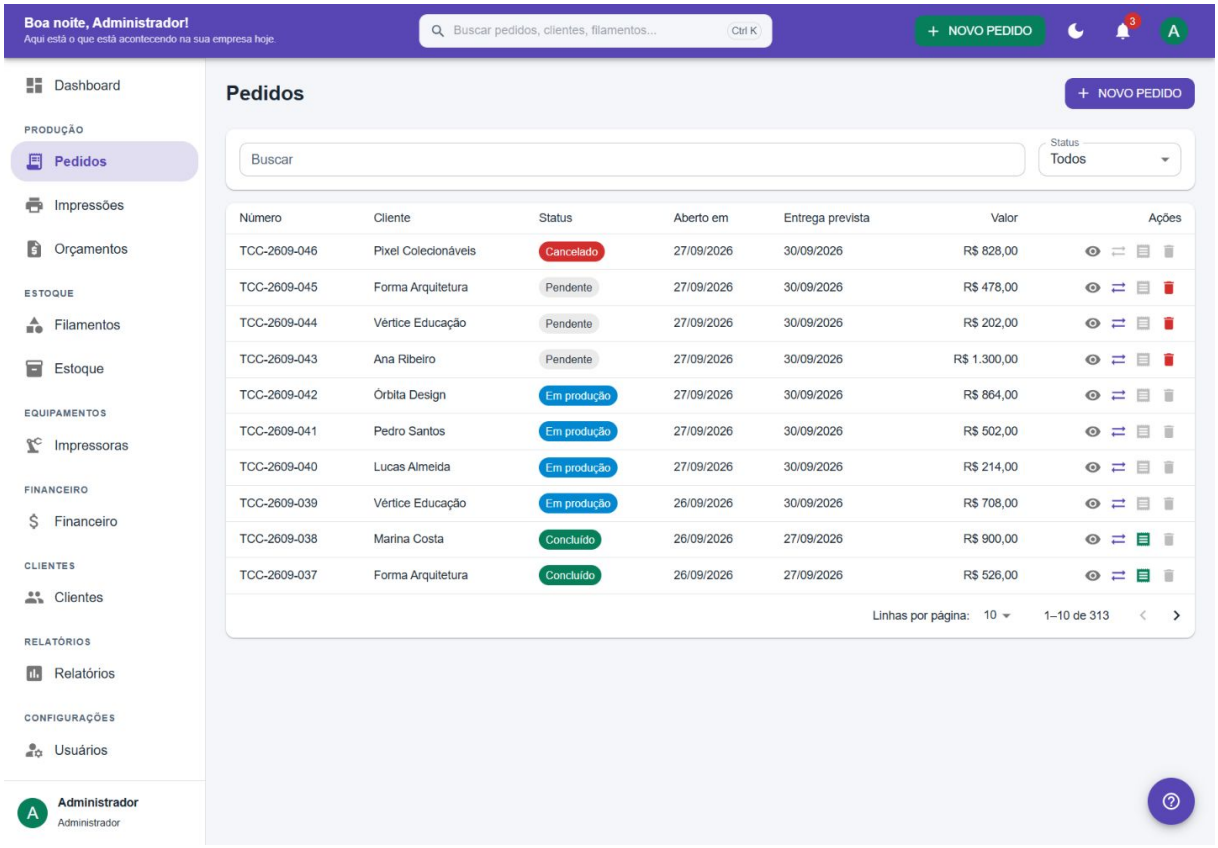
Figura 3 - Dashboard do Print3D Manager ERP




Fonte: captura do sistema com dados fictícios para demonstração, 2026.

O dashboard reúne indicadores de pedidos, produção, estoque e financeiro. A tela mostra o resultado da integração entre módulos, mas a apresentação de valores e gráficos não comprova, por si só, a correção de todos os cálculos. Essa verificação depende dos dados de entrada e dos testes das regras correspondentes.

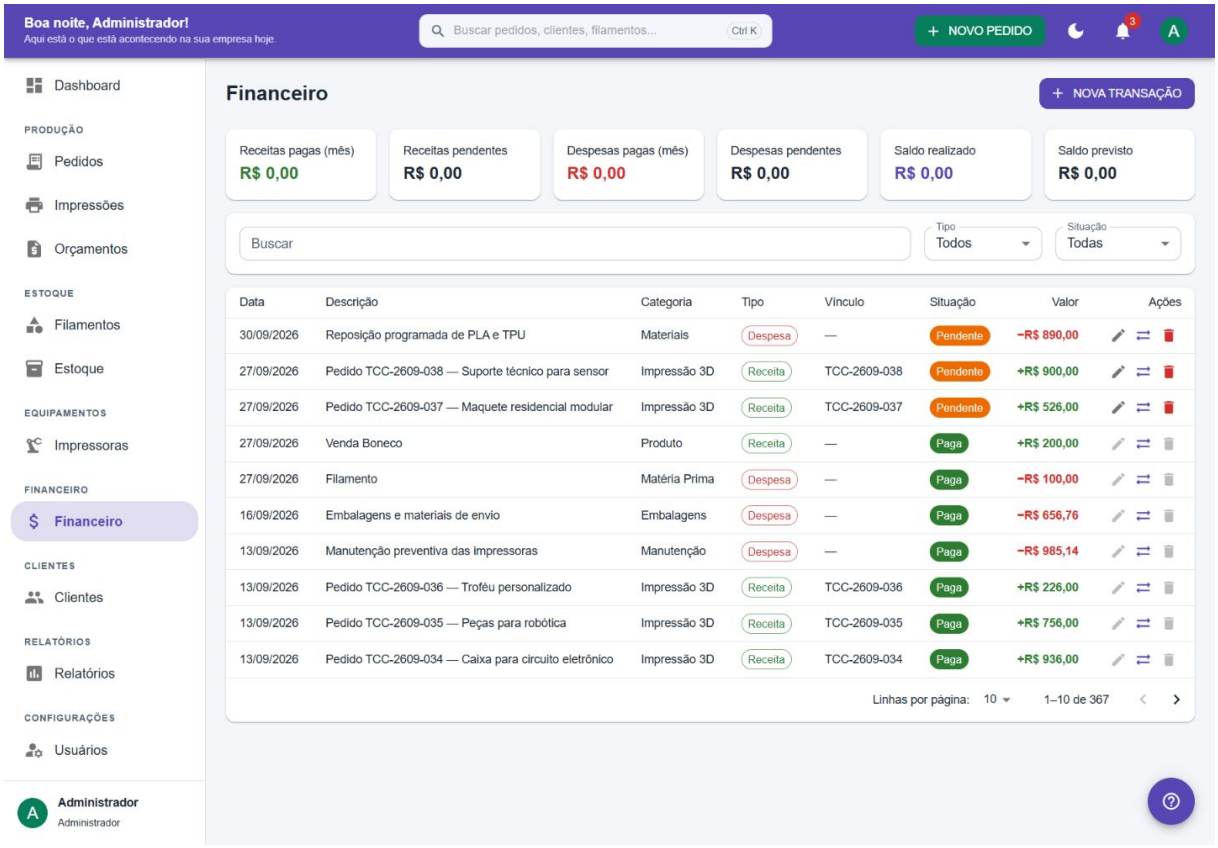
Figura 4 - Gerenciamento de pedidos




Fonte: captura do sistema com dados fictícios para demonstração, 2026.

A listagem de pedidos permite consultar cliente, estado, prazo e valor, além de filtrar os registros. O resultado visual evidencia a funcionalidade disponibilizada ao usuário. As condições permitidas para cada transição são tratadas no backend e precisam ser examinadas separadamente da aparência da tela.

Figura 5 - Módulo financeiro




Fonte: captura do sistema com dados fictícios para demonstração, 2026.

O módulo financeiro apresenta receitas, despesas e saldos, com filtros para consultar as transações. O vínculo com pedidos aproxima o registro financeiro do fluxo de produção. Essa integração também tornou relevante a análise de concorrência do faturamento, discutida a seguir.

## 4.2 Auditoria especializada

O relatório .claude/audits/FINAL-PLAN.md consolida uma rodada de auditoria em nove domínios. O documento registra 33 achados e os organiza por severidade. Esse número corresponde a itens consolidados, incluindo observações informativas; não equivale a 33 falhas exploradas nem a 33 correções realizadas. Os doze papéis configurados também não correspondem a doze relatórios especializados nessa rodada.

Na mesma rodada, o relatório registra 104 testes de backend executados sem falhas ou erros. Ainda assim, aponta cenários concorrentes ausentes da suíte. A coexistência desses registros é um resultado relevante: o conjunto de testes existente verificava determinados comportamentos, enquanto a auditoria procurou situações que não estavam representadas nas asserções.

### 4.3 Concorrência no faturamento

Um dos achados críticos descreve o risco de duplicação de receita. Segundo o relatório, o fluxo verificava a existência de faturamento e, depois, criava a transação, sem uma garantia suficiente para impedir que duas execuções atravessassem essa sequência ao mesmo tempo. Esse intervalo caracteriza um problema de verificação seguida de uso, referido no relatório como TOCTOU.

O caso reuniu perspectivas diferentes. A análise de ERP apontou o comportamento de negócio; a de banco identificou a ausência da restrição correspondente; e a de QA registrou a falta de testes de concorrência. O resultado documentado foi uma recomendação de proteção transacional e de unicidade, acompanhada de teste concorrente. O relatório não deve ser utilizado, sozinho, como prova de que essas correções foram aplicadas ou de que o risco permanece no código atual.

Esse episódio ajuda a explicar a contribuição dos agentes: o problema foi descrito por sua consequência concreta, a duplicação de uma receita, e relacionado a mecanismos técnicos e lacunas de teste. A quantidade de agentes envolvidos não constitui a evidência principal; o que sustenta a análise é a relação entre cenário, código examinado e garantia ausente.

### 4.4 Divergência entre análises

Outro episódio envolve o rate limiting, mecanismo que restringe a quantidade de requisições. A consolidação registra avaliações diferentes de segurança e infraestrutura sobre a exposição direta do backend e o tratamento do cabeçalho X-Forwarded-For. Uma análise inicial não havia reconhecido o cenário como um desvio efetivo do controle; outra examinou o comportamento da cadeia de proxy.

Conforme o relatório, a verificação registrou 13 requisições com cabeçalhos forjados sem o bloqueio esperado, em contraste com o cenário de referência que bloqueava a partir da 11ª requisição. A consolidação reconciliou os pareceres e classificou o caso como crítico. Esses números são provenientes da auditoria preservada, e não de uma nova execução realizada para esta revisão do texto.

O episódio mostra por que um parecer não pode ser aceito apenas pelo papel atribuído ao agente. A revisão de severidade foi fundamentada no cenário testado. Também mostra uma limitação da especialização: agentes podem partir de pressupostos diferentes sobre o mesmo ambiente e chegar a conclusões incompatíveis até que esses pressupostos sejam explicitados.

### 4.5 Discussão

As evidências do caso apontam utilidade na organização das instruções, na consulta ao contexto e na revisão por especialidade. Os arquivos tornam visíveis as responsabilidades previstas, e os relatórios permitem examinar perguntas que foram feitas ao sistema. O arquivo de contexto também documenta decisões que podem ser retomadas em sessões posteriores. Não foi medida, porém, a quantidade de esforço economizada por essa organização.

Os resultados dialogam com a divisão de papéis discutida por Hong et al. (2024) e Qian et al. (2024), mas a implementação do projeto possui limites próprios. Ela foi baseada em subagentes e documentos de um repositório, com supervisão humana. Não reproduziu os experimentos dos frameworks citados nem comparou, de forma controlada, diferentes estratégias de uso de IA.

Há limitações adicionais: trata-se de um único projeto, os registros de conversas não são completos e os relatórios refletem rodadas específicas. Não foram coletadas medidas sistemáticas de tempo, custo ou taxa de defeitos, nem realizada avaliação formal com usuários da área de impressão 3D. As telas com dados fictícios demonstram a interface; não validam a aplicação em uma empresa. Portanto, as conclusões se restringem ao processo e às evidências descritas.

## 5 CONCLUSÃO

O Print3D Manager ERP foi desenvolvido como aplicação web para integrar atividades de empresas de impressão 3D. O estudo descreveu o uso de doze subagentes configurados no Claude Code e examinou sua participação a partir dos prompts persistentes, do contexto do projeto e dos relatórios de auditoria.

No caso analisado, os agentes apoiaram a distribuição de tarefas e a revisão de problemas que atravessavam diferentes partes da aplicação. Os episódios de faturamento concorrente e de reavaliação do rate limiting mostram que a contribuição foi além da produção de código: incluiu explicitar cenários de falha, relacionar evidências e contestar conclusões anteriores.

A questão de pesquisa pode ser respondida dentro desses limites. Agentes especializados podem apoiar o desenvolvimento quando recebem contexto, escopo e critérios de verificação, mas a configuração não garante a execução correta dessas instruções. A aceitação depende de evidências e da análise do desenvolvedor. O estudo não demonstrou ganho de produtividade nem permite generalizar os resultados para outros projetos.

Como continuidade, propõe-se registrar invocações, versões dos modelos, comandos e resultados de forma sistemática; medir tempo e custo; e comparar tarefas equivalentes com diferentes estratégias de uso de IA. Uma avaliação com profissionais de impressão 3D também permitiria examinar a adequação do produto ao trabalho real das empresas.

## REFERÊNCIAS BIBLIOGRÁFICAS

ANTHROPIC. Create custom subagents. Claude Code Docs, 2026. Disponível em: https://code.claude.com/docs/en/sub-agents. Acesso em: 14 out. 2026.

BASS, L.; CLEMENTS, P.; KAZMAN, R. Software Architecture in Practice. 4. ed. Boston: Addison-Wesley Professional, 2021.

DATE, C. J. Introdução a sistemas de bancos de dados. 8. ed. Rio de Janeiro: Elsevier, 2004.

ELMASRI, R.; NAVATHE, S. B. Fundamentals of Database Systems, Global Edition. 7. ed. Harlow: Pearson, 2016. Disponível em: https://www.pearson.com/en-gb/subject-catalog/p/Elmasri-Fundamentals-of-Database-Systems-Global-Edition-7th-Edition/P200000004142. Acesso em: 14 out. 2026.

FOWLER, M. Patterns of Enterprise Application Architecture. Boston: Addison-Wesley, 2002.

GAMMA, E.; HELM, R.; JOHNSON, R.; VLISSIDES, J. Padrões de projeto: soluções reutilizáveis de software orientado a objetos. Porto Alegre: Bookman, 2000.

HONG, S. et al. MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework. In: INTERNATIONAL CONFERENCE ON LEARNING REPRESENTATIONS, 12., 2024. Proceedings [...]. 2024. Disponível em: https://arxiv.org/abs/2308.00352. Acesso em: 14 out. 2026.

PENG, S.; KALLIAMVAKOU, E.; CIHON, P.; DEMIRER, M. The Impact of AI on Developer Productivity: Evidence from GitHub Copilot. arXiv, 2023. DOI: 10.48550/arXiv.2302.06590. Disponível em: https://arxiv.org/abs/2302.06590. Acesso em: 14 out. 2026.

PRESSMAN, R. S. Engenharia de software: uma abordagem profissional. 7. ed. Porto Alegre: AMGH, 2011.

QIAN, C. et al. ChatDev: Communicative Agents for Software Development. In: ANNUAL MEETING OF THE ASSOCIATION FOR COMPUTATIONAL LINGUISTICS, 62., 2024, Bangkok. Proceedings [...]. Bangkok: Association for Computational Linguistics, 2024. p. 15174-15186. DOI: 10.18653/v1/2024.acl-long.810. Disponível em: https://aclanthology.org/2024.acl-long.810/. Acesso em: 14 out. 2026.

SOMMERVILLE, I. Engenharia de software. 9. ed. São Paulo: Pearson Prentice Hall, 2011.

STALLINGS, W. Cryptography and Network Security: Principles and Practice. 7. ed. Boston: Pearson, 2017.